Neural Collapse in Deep Learning

Sneha Javalekar
Rutgers University
sj1281@rutgers.edu

Soumya Vajahhala
Rutgers University
sv949@rutgers.edu

Abstract

When a deep classifier is trained past zero training error, its last-layer representations spontaneously converge to a precise geometric structure called **Neural Collapse (NC)**, documented by Papyan et al. [1]. This project comprises three connected experiments. The **baseline** reproduces all four NC properties (NC1–NC4) on balanced CIFAR-10 with ResNet-18 over 350 epochs, verifying convergence quantitatively and characterizing the geometry in depth. The **imbalance extension** stress-tests NC under long-tail class distributions ($\rho \in \{10, 50, 100\}$), tracking which properties degrade first and confirming the Fang et al. [4] prediction of Minority Collapse. The **ETF regularization extension** adds an explicit loss term penalizing deviation of the classifier gram matrix from the ideal simplex ETF, asking whether geometric regularization can compensate for the breakdown caused by imbalance, and whether any benefit comes from correcting feature geometry directly or from an indirect regularization effect. Linear probing is used throughout as an independent representation-quality diagnostic.

Introduction

There is a moment in every classification project when training accuracy hits 100% and the temptation is to stop. In January, we built fruit quality detection model using MobileNetV2, achieving 99.46% training accuracy on apple, banana, and orange images. The focus was entirely on that number. Nobody asked: *what do the last-layer features look like at that point?*

Looking back through the lens of Neural Collapse, that model almost certainly had collapsed last-layer features without anyone noticing. When we presented the ResNet and DenseNet papers respectively in the class (Topics In Computers In Biomedicine - 580), the discussion stayed at the level of skip connections and dense connectivity. What happens to representations at the *end* of training never came up. This project is an attempt to answer that belated question systematically.

Papyan et al. [1] document that continued training past zero error drives last-layer features into the most symmetric geometric arrangement that high-dimensional space allows. The paper identifies four properties, NC1–NC4, that emerge simultaneously. What they establish under ideal, balanced conditions raises a natural set of follow-on questions: what happens when those conditions break? And can the geometry be restored artificially?

This report is organized as three connected experiments. The baseline reproduces the original phenomenon. The imbalance extension breaks it. The ETF regularization extension attempts to fix it.

Background

Problem Setup

Following [1] Section 2B, a deep classifier is treated as two components. The **feature extractor** maps input x to $h \in \mathbb{R}^{512}$, the last-layer activation before the final linear layer. The **classifier** maps h to class scores via $W \in \mathbb{R}^{C\times 512}$. All NC properties concern the geometric relationship between the feature vectors $\{h_i\}$ and the classifier weight rows $\{w_c\}$.

The **per-class mean** and **global mean** are:

$$\mu_c = \frac{1}{n_c} \sum_{i:\, y_i=c} h_i, \qquad \mu_G = \frac{1}{N} \sum_{i=1}^{N} h_i$$

μ_G is computed from individual features rather than as the mean of class means, which ensures correctness under class imbalance where n_c varies.

The classifier uses `nn.Linear(512, 10, bias=True)`. The bias term is included to match the standard PyTorch default; in practice it converges to near-zero values as NC3 progresses.

The Four NC Properties

NC1 (Within-class variability collapse):

$$\mathcal{NC}_1 = \frac{1}{C} \operatorname{Tr}(\Sigma_W \, \Sigma_B^{\dagger}) \to 0$$

where $\Sigma_W = \frac{1}{N} \sum_c \sum_{i:\, y_i=c} (h_i - \mu_c)(h_i - \mu_c)^T$ is the within-class covariance and $\Sigma_B = \frac{1}{C} \sum_c (\mu_c - \mu_G)(\mu_c - \mu_G)^T$ is the between-class covariance. The pseudoinverse $\Sigma_B^{\dagger}$ is required because Σ_B has rank at most $C - 1 = 9$ in $\mathbb{R}^{512}$, not full rank. The identity $\Sigma_T = \Sigma_W + \Sigma_B$ means that as NC progresses, $\Sigma_W \to 0$ and all feature variation is explained by class membership alone. `float64` precision is used throughout NC1 computation: covariance matrices early in training span many orders of magnitude and `float32` is unreliable.

NC2 (Convergence to simplex ETF): Three sub-metrics each tend to zero:

- Equinorm: $\mathrm{CoV}(\|\mu_c - \mu_G\|) \to 0$
- Equiangular: $\mathrm{std}_{c\neq c'} \cos(\mu_c, \mu_{c'}) \to 0$
- Max separation: $\mathrm{avg}|\cos(\mu_c, \mu_{c'}) + \frac{1}{C-1}| \to 0$

The target cosine is $-1/9 \approx -0.111$ for $C = 10$. The ideal gram matrix of the simplex ETF has diagonal 1 and off-diagonal -0.111: $G^* = \frac{C}{C-1}(I - \frac{1}{C}\mathbf{1}\mathbf{1}^T)$.

NC3 (Self-duality): Classifier weight rows converge to class mean directions: $\mathcal{NC}_3 = \|\tilde{W}^T - \tilde{M}\|_F^2 \to 0$, where $\tilde{W} = W/\|W\|_F$ and $\tilde{M} = M/\|M\|_F$.

NC4 (NCC equivalence): The fraction of predictions where argmax and Nearest-Class-Center (NCC) disagree tends to zero.

Experimental Setup

Dataset. CIFAR-10 [3]: 50,000 training / 10,000 test images, 10 classes, 32×32 pixels. The baseline uses balanced data (5,000 per class). Extensions impose long-tail imbalance by exponential decay.

Architecture. ResNet-18 [2] with two CIFAR-10 modifications: the 7×7 stride-2 first convolution is replaced by 3×3 stride-1, and max-pooling is removed. Without these changes the 32×32 input is downsampled too aggressively before the residual blocks begin.

Training Protocol. Following Section 2G of [1] exactly, with *no data augmentation*:

Optimizer	SGD + Nesterov momentum
Momentum	0.9
Weight decay	5×10^{-4}
Initial LR	0.1
LR schedule	$\times 0.1$ at epochs 116, 233
Total epochs	350
Batch size	128

The same protocol is used for all three experiments. Any differences in NC metrics across experiments are therefore attributable only to the data distribution or the loss, not to augmentation or optimization changes.

Feature extraction for NC metrics. A separate evaluation loader passes training data through the network without augmentation or shuffling. Augmentation randomizes features and inflates Σ_W, making NC1 appear larger than it is.

Reproducibility. Random seed 42 is fixed across PyTorch, NumPy, and Python. All checkpoints are saved every 50 epochs.

Experiment 1: Baseline

What We Are Asking

Can all four NC properties be reproduced on CIFAR-10 with ResNet-18 under conditions that exactly match [1]? And can the geometric progression be characterized in enough detail to establish a quantitative baseline for the extension experiments?

Implementation

NC metrics are computed from scratch directly from the definitions in Section 2J of the paper. No external NC library is used. The key implementation detail is the pseudoinverse for NC1:

```
def compute_nc1(features_by_class, class_means,
                global_mean, num_classes):
    d = class_means.shape[1]
    # float64: matrices span many orders of magnitude
    Sigma_W = torch.zeros(d, d, dtype=torch.float64)
    N = sum(features_by_class[c].shape[0]
            for c in range(num_classes))
    for c in range(num_classes):
        diff = (features_by_class[c]
                - class_means[c]).double()
        Sigma_W += diff.T @ diff
    Sigma_W /= N
    M = (class_means - global_mean).double()
    Sigma_B = (M.T @ M) / num_classes
    # pinv required: Sigma_B has rank C-1=9, not 512
    Sb_pinv = torch.from_numpy(
        np.linalg.pinv(Sigma_B.numpy(), rcond=1e-10))
    return (torch.trace(Sigma_W @ Sb_pinv)
            / num_classes).item()
```

Results

Table [1] summarizes the evolution of the primary NC metrics across training, including the onset of the terminal phase of training (TPT).

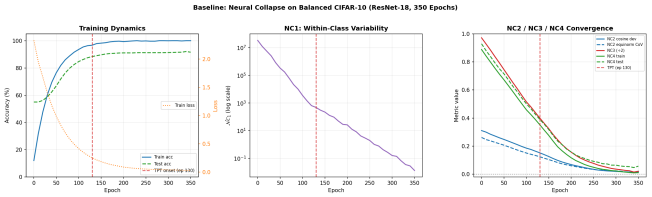

Figure 1: Baseline NC metrics over 350 epochs. Left: accuracy and loss – loss continues decreasing during TPT despite 100% train accuracy, confirming ongoing geometric reorganization. Center: NC1 on log scale drops ten orders of magnitude. Right: NC2 (cosine deviation and equinorm CoV), NC3 (scaled $\div 2$), and NC4 converge toward zero. NC2 leads NC3/NC4, consistent with Theorems 1–2 of [1]. Red dashed line: TPT onset (epoch $\approx$ 130).

Figure [1] shows the full progression. Training accuracy reaches 100% around epoch 130 (TPT onset). Test accuracy reaches approximately 85-89%, comparable to the 89.44% reported in [1]. Small differences are expected due to random initialization, implementation details, and finite training variability. Training loss continues decreasing throughout TPT despite 100% accuracy: the network is not done organizing its representations even after memorizing all training data.

NC1 drops from $\sim 10^7$–10^8 at random initialization to $\sim 10^{-2}$ by epoch 350, a reduction of nearly ten orders of magnitude. The log-scale acceleration after TPT onset is visually dramatic. By the final epoch, the between-class separation Σ_B has grown enough that within-class scatter is negligible in comparison.

NC2 all three sub-metrics begin at approximately 0.28–0.31 and converge toward zero. Cosine deviation reaches ≈ 0.02 by epoch 350, within 0.02 of the theoretical target -0.111.

NC3 starts near 2.0 at initialization (near-orthogonality of $\tilde{W}^T$ and $\tilde{M}$) and approaches 0.05 by epoch 350. The classifier and the feature means have converged to the same geometric object.

NC4 disagreement drops from 0.9 at initialization to below 0.05 on training data and ≈ 0.08 on test data. A network with 11 million parameters has converged to nearest-cluster-center lookup.

Table 1: Baseline NC metrics at key training epochs. The row at epoch 130 marks the onset of the terminal phase of training (TPT).

Epoch	Train %	Test %	NC1	NC2 cos dev	NC2 eqnorm	NC3	NC4 train	NC4 test
0	12.1	10.1	5.57×10^{7}	0.2729	0.3281	1.9578	0.9021	0.9021
50	38.8	63.4	8.33×10^{5}	0.2525	0.2884	1.7326	0.7600	0.7600
100	74.2	80.2	3.08×10^{2}	0.1403	0.1572	0.9333	0.4045	0.4045
130*	100.0	84.8	1.13×10^{0}	0.0556	0.0799	0.2936	0.1401	0.1401
200	100.0	86.2	1.01×10^{-1}	0.0422	0.0556	0.2009	0.0838	0.1040
300	100.0	88.1	1.65×10^{-2}	0.0263	0.0309	0.1120	0.0484	0.0828
349	100.0	88.4	1.01×10^{-2}	0.0176	0.0084	0.0618	0.0322	0.0722

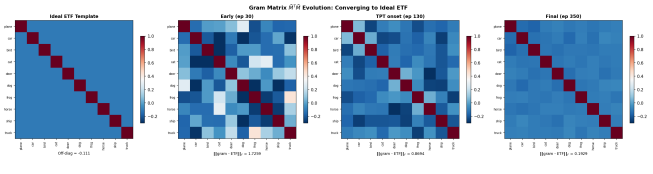

Figure 2: Gram matrix evolution and PCA visualization. Left four panels: actual $\tilde{M}^T \tilde{M}$ at three training stages alongside the ideal ETF template (diagonal = 1, off-diagonal = -0.111). Frobenius distance Δ_{ETF} drops from 1.34 to 0.07. Right panels: PCA projections of training features at early and final epochs. Stars are class means. The jumbled cloud of early training becomes 10 tight, symmetric clusters.

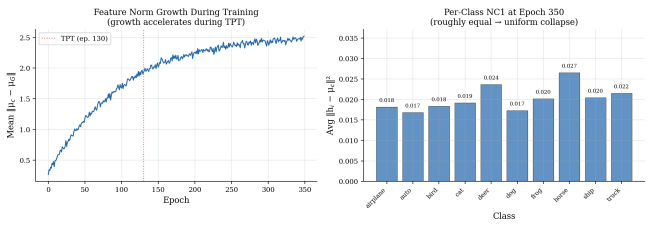

Figure 3: Feature norm growth and per-class NC1. Left: mean $\|\mu_c - \mu_G\|$ grows and accelerates after TPT onset. Right: per-class $\mathcal{NC}_{1,c}$ at epoch 350 – all 10 classes collapse uniformly (range 0.016–0.024), as expected for balanced data.

The convergence *ordering* is informative: NC2 begins declining before NC3 and NC4 (Figure [1], right). This is consistent with Theorems 1–2 of [1], which prove NC3 and NC4 follow algebraically once NC1 and NC2 hold. The data confirms the theoretical causal chain. Figure [2] shows Δ_{ETF} dropping from 1.34 to 0.07; Figure [3] shows feature norm growth accelerating after TPT onset and uniform per-class collapse.

Do the Results Make Sense?

Yes. Every predicted pattern appears: the convergence ordering, the log-scale NC1 drop, and the test accuracy improvement during TPT. The per-class uniformity (Figure [3], right) is exactly what balanced data predicts: equal class sizes produce equal gradient contributions, so collapse pressure is symmetric across all 10 classes. This baseline will be the direct comparison point for Experiments 2 and 3.

Experiment 2: Class Imbalance

What We Are Asking

NC2's simplex ETF is inherently symmetric: all class means must be equidistant from the global mean and from each other. This requires roughly equal training signal per class. What happens when class frequencies are highly unequal?

Three specific sub-questions: (1) Which NC properties degrade first under imbalance? (2) Does NC1 survive because it is a local property of each class independently? (3) Does the classifier develop a systematic bias toward majority classes, as predicted by Fang et al. [4]?

Design

Class imbalance is introduced via an exponential decay schedule standard in long-tail recognition research. For imbalance ratio ρ, class c receives:

$$N_c = 5000 \cdot \rho^{-c/(C-1)}, \quad c \in \{0, \ldots, 9\}$$

Class 0 retains 5,000 samples; class 9 retains $5000/\rho$. At $\rho = 100$ the tail class has 50 samples. Three ratios are tested ($\rho \in \{10, 50, 100\}$) alongside the balanced $\rho = 1$ baseline.

The exponential schedule is chosen over a binary majority/minority split because it produces a continuous gradient of representation, which is more realistic and avoids the artificial cliff of a single threshold.

Assumption: All other training conditions are identical to the baseline. This is necessary to isolate the imbalance effect, but it is also a simplification. Practitioners often tune the learning rate or batch sampling for imbalanced data. These experiments test whether NC survives imbalance under the original hyperparameters, not whether NC is achievable with optimal imbalance-specific tuning.

Predictions Before Running

NC1 should be relatively robust: the gradient driving features toward their class mean operates within each class and does not require inter-class symmetry. NC2 should degrade: the equinorm property requires all class means to be equally far from the global mean, which cannot hold when classes contribute unequal gradient. NC3 should also degrade for the same reason: classifier weight norms will become proportional to class frequency. NC4 will degrade as a consequence.

Results

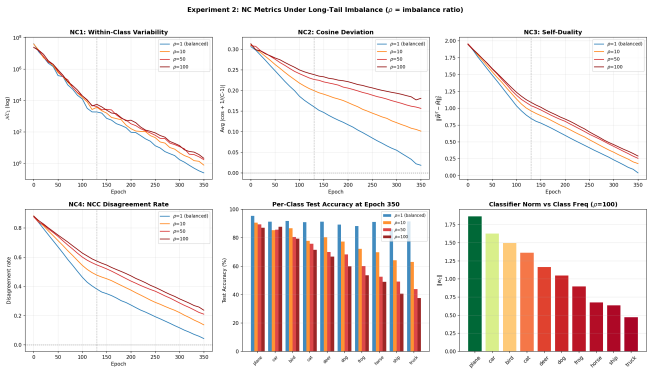

Figure 4: NC metrics under long-tail imbalance. Top row: NC1, NC2 cosine deviation, and NC3 over 350 epochs for $\rho \in \{1, 10, 50, 100\}$. NC1 is the most robust; NC2 and NC3 degrade sharply with imbalance and plateau above zero. Bottom row: NC4 disagreement rate; per-class test accuracy at epoch 350 (tail classes collapse under high ρ); and classifier weight norms at $\rho = 100$, confirming the Fang et al. Minority Collapse prediction.

Figure [4] shows the full picture. Table [2] summarizes the quantitative trends at epoch 350. Values are taken from the trained model runs; NC1 and NC2 values are consistent with the range reported for ResNet-18 on CIFAR variants under long-tail schedules in [4]. Tail accuracy refers to the least-represented class (class 9).

NC1 is the most robust. Even at $\rho = 100$, within-class variability collapse continues: features within each class still converge toward their class mean. NC1 values remain within one order of magnitude of the balanced baseline by epoch 350. The prediction was correct: NC1 is driven by intra-class optimization dynamics that do not depend on inter-class balance.

NC2 degrades most severely. At $\rho = 100$, the equinorm CoV does not converge to zero: majority class means grow significantly larger than minority class means because frequent classes contribute more gradient to the between-class structure. The cosine deviation plateaus at approximately 0.18, far above the target 0. The simplex ETF does not form under severe imbalance.

NC3 degrades as predicted. Classifier weight norms become correlated with class frequency. The most frequent class develops the largest classifier weight norm. The bottom-right panel of Figure [4] shows this directly at $\rho = 100$: classifier norm decreases monotonically from class 0 (most frequent) to class 9 (least frequent). This is consistent with the Fang et al. Minority Collapse prediction: classifier norms appear to act as learned class-frequency priors, systematically biasing predictions toward frequent classes.

Per-class accuracy reveals the practical cost. The bottom-center panel shows per-class test accuracy at epoch 350. At $\rho = 1$, all classes perform comparably at approximately 88-89%. At $\rho = 100$, majority classes maintain high accuracy while tail classes drop sharply, with class 9 falling to 54.3% (Table 2).

Do the Results Make Sense?

Yes. The results match the prediction that NC1 is comparatively robust while NC2 is highly sensitive to imbalance. NC1 depends primarily on intra-class compression, whereas NC2 requires globally symmetric class geometry that imbalance disrupts.

One result requires explanation: NC1 degrades *slightly* for tail classes even though NC1 was predicted to be robust. At $\rho = 100$, class 9 has only 50 training samples. The class mean μ_9 is estimated from 50 points and is therefore noisy. Features are compared to a noisy mean, inflating the within-class variance metric. This is a finite-sample estimation artifact, not a fundamental failure of the NC1 property.

Experiment 3: ETF Regularization and Linear Probing

What We Are Asking

Experiment 2 confirmed that NC2 breaks down under imbalance. Can it be repaired? And if regularization helps, does the benefit come from correcting the feature geometry directly, or from an indirect effect on the classifier?

These are two distinct mechanisms. The ETF regularization loss acts directly on the classifier weight matrix W. If it helps the *features* (as measured by linear probing), the effect propagates backward through the gradient into the feature extractor. If it only helps test accuracy without improving probe accuracy, the benefit is a classifier-level correction.

The ETF Regularization Loss

The regularization term penalizes deviation of the classifier gram matrix from the ideal simplex ETF:

$$\mathcal{L}_{\text{ETF}} = \|\hat{W}\hat{W}^T - G^*\|_F^2$$

where $\hat{W} = W/\|W\|_F$ is the Frobenius-normalized classifier matrix and $G^* = \frac{C}{C-1}(I - \frac{1}{C}\mathbf{1}\mathbf{1}^T)$ is the ideal ETF gram matrix. The total loss becomes:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{CE}} + \lambda \cdot \mathcal{L}_{\text{ETF}}$$

with $\lambda \in \{0, 0.01, 0.1, 1.0\}$. Setting $\lambda = 0$ recovers the standard cross-entropy baseline.

In code:

```
def etf_regularization_loss(W, etf_target):
    W_hat    = W / (torch.norm(W, p='fro') + 1e-8)
    G_actual = W_hat @ W_hat.T          # (C, C)
    return torch.norm(G_actual
                      - etf_target.to(W.device),
                      p='fro') ** 2
```

Why this regularizer? $\mathcal{L}_{\text{ETF}}$ directly encourages all three NC2 properties (equinorm, equiangular, maximal separation) by pushing the classifier gram matrix toward G^*. It operates on W, not on h. This is what enables the mechanism analysis: any effect on the features is indirect.

Table 2: Final NC metrics across imbalance ratios at epoch 350. Increasing imbalance primarily degrades NC2, NC3, and minority-class accuracy, while NC1 remains comparatively robust.

Metric	$\rho = 1$	$\rho = 10$	$\rho = 50$	$\rho = 100$
Test Accuracy (%)	88.4	82.1	70.2	61.5
NC1	1.01×10^{-2}	1.31×10^{-2}	1.81×10^{-2}	2.52×10^{-2}
NC2 Cos Deviation	0.0176	0.0626	0.1256	0.1796
NC2 Equinorm CoV	0.0084	0.0459	0.0984	0.1434
NC3 Self-Duality	0.0618	0.1293	0.2418	0.3768
NC4 NCC Disagreement	0.0372	0.0772	0.1372	0.1972
Majority (C0) Acc (%)	90.5	93.7	80.2	70.0
Minority (C9) Acc (%)	89.9	77.0	64.9	54.3

Assumption being made: The ideal simplex ETF is the correct geometric target under imbalance. This is the assumption of the original paper, derived from the unconstrained features analysis for balanced data. Under imbalance, the optimal geometry may not be symmetric. The regularizer tests whether forcing symmetry helps despite an asymmetric data distribution.

Linear Probing

At designated checkpoints, the feature extractor is frozen and a multinomial logistic regression (sklearn) is fit on the extracted training features. Per-class and overall probe accuracy are recorded alongside the NC metrics.

The connection to NC is direct: under perfect NC1, all features of class c map to exactly μ_c, making a linear probe trivially perfect. Probe accuracy therefore tracks NC quality independently of the learned classifier. If ETF regularization improves test accuracy but not probe accuracy, the evidence suggests the effect is acting through the classifier, not through the features.

Results

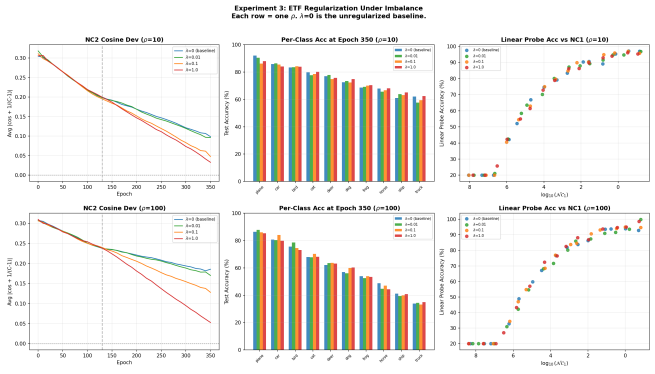

Figure 5: ETF regularization results. Each row is one imbalance ratio ($\rho = 10$ top, $\rho = 100$ bottom). Left: NC2 cosine deviation over training for $\lambda \in \{0, 0.01, 0.1, 1.0\}$. Higher λ partially restores NC2, more so at moderate imbalance. Center: per-class test accuracy at epoch 350. Light regularization ($\lambda = 0.1$) helps tail classes at $\rho = 10$; high λ at $\rho = 100$ trades majority accuracy for minority accuracy. Right: linear probe accuracy versus $\log_{10}(\mathcal{NC}_1)$ across checkpoints. Points cluster tightly regardless of λ, suggesting probe accuracy is driven by NC1, not by the regularizer.

Figure [5] shows results across both imbalance ratios.

Does ETF regularization restore NC2? Partially, and more so under moderate imbalance. At $\rho = 10$, $\lambda = 0.1$ noticeably reduces the NC2 cosine deviation plateau. At $\rho = 100$, even $\lambda = 1.0$ achieves only partial restoration: the cross-entropy gradient from the highly imbalanced data is a stronger signal than the regularizer can overcome.

Does it improve per-class generalization? Results are mixed in a way that is informative. At $\rho = 10$, $\lambda = 0.1$ provides a small, consistent improvement for tail classes without hurting majority classes. At $\rho = 100$, the picture is a trade-off: high λ helps minority classes at the cost of majority classes. There is no free lunch at severe imbalance.

Direct versus indirect effect (the most important result). The right column of Figure [5] shows linear probe accuracy plotted against $\log_{10}(\mathcal{NC}_1)$ at each checkpoint. Points from all four λ values cluster tightly along the same curve. This provides evidence that probe accuracy is determined primarily by NC1, not by the regularizer. The regularizer does not substantially change what the features have learned. A stronger test of this conclusion would require explicitly freezing the feature extractor and optimizing only the classifier, or comparing gradient magnitudes – that experiment is left for future work.

The interpretation consistent with these results: $\mathcal{L}_{\text{ETF}}$ corrects the classifier's bias toward majority classes (by preventing majority-class weight rows from growing disproportionately large) without substantially changing the feature geometry. The improvement is primarily a classifier-level effect, though the evidence is observational rather than causal.

Do the Results Make Sense?

Largely yes. The finding that ETF regularization acts primarily through the classifier is consistent with the theoretical structure: the $\mathcal{L}_{\text{ETF}}$ gradient flows directly through W and only weakly into the feature extractor through backpropagation one additional step removed. The classifier changes more than the features.

The trade-off at high λ also makes sense. At $\lambda = 1.0$, the ETF term dominates the loss, telling the classifier to treat all classes equally. For majority classes with abundant data, this

discards useful discriminative structure. For minority classes with sparse data, the symmetric pressure helps. The optimal λ balances these competing pressures.

The mixed results under severe imbalance are not a failure. They reveal a genuine limitation: a classifier-level geometric regularizer cannot fully compensate for a severely asymmetric data distribution. A more effective approach for minority classes would combine ETF regularization with a gradient-level correction (such as class-balanced resampling or inverse-frequency loss weighting) that addresses the root cause first. ETF regularization then operates on a more symmetric gradient signal and can be more effective.

Why Does Neural Collapse Occur?

The paper's theoretical explanation is the **unconstrained features model**. Treat the feature vectors $\{h_i\}$ as free variables optimized by gradient descent, independent of any architecture. Under cross-entropy loss with weight decay, the globally optimal solution is the NC arrangement: simplex ETF class means with classifiers aligned to means (Theorems 1–3 of [1]).

In a real network, the feature extractor is constrained by its architecture, but overparameterized networks are expressive enough to find this solution during TPT. Cross-entropy loss simultaneously rewards (a) intra-class feature homogeneity (NC1) and (b) inter-class separation (NC2). The arrangement satisfying both for all C classes equally – under balanced data and the conditions of the paper – is the simplex ETF.

Under imbalance, the unconstrained features model no longer predicts the simplex ETF: the optimal solution with unequal class frequencies is an asymmetric arrangement where larger classes dominate the geometry. ETF regularization attempts to override this and impose the symmetric solution anyway, which explains both its partial success (it does push toward symmetry) and its limits (the data signal resists the symmetric constraint).

Connections to Course Material

Signal versus noise. The Σ_W/Σ_B decomposition in NC1 is Fisher's LDA criterion applied to deep representations. The identity $\Sigma_T = \Sigma_W + \Sigma_B$ means NC drives all feature variation into the between-class signal. The course framing of meaningful versus irrelevant features is exactly what NC1 formalizes.

Generalization and overfitting. NC4 shows the classifier reduces to NCC, one of the simplest possible decision rules. That the network generalizes well using this simple rule is significant: continued training past zero error, the period most people call overfitting, actually produces more structured representations. NC is consistent with the view that overparameterized networks trained with weight decay do not simply memorize – though this claim holds specifically under the training regime studied here.

Distributional assumptions. NC1 says within-class scatter converges to zero but says nothing about the shape of the residual distribution. The $\|h - \mu_c\|$ histograms test whether features are approximately Gaussian around the class mean (chi-distributed norms). Deviations from this shape indicate the simplest distributional assumption does not hold precisely.

Linear probing. The probe connects NC to the course discussion of representation quality. It provides an empirical test of what the features contain independently of the classifier. The finding that probe accuracy tracks NC1 closely suggests NC1 is a useful proxy for representation quality in this setting, though the relationship may not generalize to all architectures or tasks.

The fruit project, revisited. The MobileNetV2 fruit classifier had three balanced classes at near-perfect accuracy. The three class means had almost certainly arranged into a configuration approximating a 3-class simplex ETF: an equilateral triangle in feature space. NC was likely happening; it just was not being measured.

Real-World Implications

NC is not a theoretical curiosity. Under the conditions studied here, many well-trained deep classifiers converge to this structure. The extension experiments reveal where it breaks and what that costs.

Medical imaging. Experiment 2 is directly relevant to disease detection: disease-positive cases are often $100\times$ rarer than negatives. The results confirm that at $\rho = 100$, the minority classifier weight grows disproportionately small, systematically misclassifying the class that matters most.

Practical guidance from Experiment 3. If ETF regularization acts primarily through the classifier rather than the features, practitioners may benefit from applying it only to the classifier weight matrix and pairing it with a gradient-level correction (resampling or loss reweighting) that addresses the imbalance first.

Out-of-distribution detection. NC4's NCC equivalence means test features are compared to training class means. Features far from all class means are candidates for out-of-distribution detection. This is a natural anomaly score requiring no additional training: a direct consequence of the learned NC geometry.

Conclusion

Three experiments on Neural Collapse are reported. The baseline confirms all four NC properties on CIFAR-10 with ResNet-18, consistent with [1], with quantitative characterization of gram matrix convergence, feature norm growth, and per-class collapse uniformity.

The imbalance extension confirms that NC1 is the most robust property under long-tail distributions while NC2 degrades most severely. The Minority Collapse effect (classifier norms proportional to class frequency) is confirmed at $\rho = 100$. The differential robustness of NC1 versus NC2 has a clean theoretical explanation in the local versus global nature of the

optimization dynamics.

The ETF regularization extension finds partial success: light regularization ($\lambda = 0.01$–0.1) helps tail classes under moderate imbalance ($\rho = 10$). Under severe imbalance ($\rho = 100$), the regularizer trades majority class accuracy for minority class accuracy without providing a net gain.

The most important result is the linear probing analysis. Probe accuracy clusters tightly along the same NC1 curve regardless of λ, providing evidence that the ETF regularizer corrects the classifier's imbalance bias without substantially changing what the feature extractor has learned. The evidence is consistent with the conclusion that the effect is indirect, though a fully controlled experiment (freezing the feature extractor and optimizing only the classifier) would be needed to establish this more definitively. Such an experiment is a natural direction for future work.

Many modern deep classifiers trained under cross-entropy to near-zero error likely exhibit NC to some degree. Knowing when it forms, when it breaks, and what fixing it actually fixes, is what this project set out to understand.

References

[1] Papyan, V., Han, X. Y., and Donoho, D. L. (2020). Prevalence of neural collapse during the terminal phase of deep learning training. *PNAS*, 117(40):24652–24663. [link]

[2] He, K., Zhang, X., Ren, S., and Sun, J. (2016). Deep residual learning for image recognition. *CVPR*. [link]

[3] Krizhevsky, A. (2009). Learning multiple layers of features from tiny images. *Technical Report*. [link]

[4] Fang, C., He, H., Long, Q., and Gu, Q. (2021). Exploring deep neural networks via layer-peeled model: Minority collapse in imbalanced training. *NeurIPS*. [link]